

Phụ lục A — Python cho chuỗi thời gian

Phụ lục này là **bản tra cứu** các hàm NumPy và pandas dùng nhiều nhất khi làm việc với chuỗi thời gian. Mỗi hàm có: nó làm gì (bằng lời thường), một ví dụ nhỏ, và **output thật** in ngay dưới dạng chú thích `#`. Buổi 3 dạy lại phần thời gian đầy đủ kèm lab. Các buổi khác trở về đây khi cần nhắc một cú pháp.

Người đọc cần biết Python cơ bản: biến, list, hàm, vòng `for`, `import`. Không cần biết trước pandas.

Mọi đoạn code **đã chạy** ngày 2026-09-18 trong môi trường của buổi 3. Phiên bản các thư viện:

Python	pandas	NumPy	polars	DuckDB
3.12	3.0.5	2.5.3	1.44.2	1.5.5

Riêng các dòng ghi "pandas 2.3.3" chạy trong môi trường của buổi 14. Mỗi ví dụ giả định đã có sẵn:

```
import numpy as np
import pandas as pd
```

Vì sao có hai bản pandas trong khoá? pandas 3 ra đầu năm 2026. Tới giữa tháng 9/2026, các thư viện dự báo statsforecast, utilsforecast, mlforecast và autogluon.timeseries vẫn chỉ chạy với pandas 2. Vì vậy các buổi đầu dùng pandas 3, còn buổi nào cần các thư viện đó thì dùng pandas 2.3.3. Mỗi buổi có môi trường riêng nên hai bản không đụng nhau. Chỗ khác nhau giữa hai bản nằm ở mục 9.

Mục lục

- 1. Môi trường của một buổi
- 2. NumPy tối thiểu
- 3. pandas tối thiểu và mốc thời gian
- 4. resample — đổi tần suất
- 5. Dời và cửa sổ: shift, diff, rolling, ewm
- 6. merge_asof — ghép với giá trị gần nhất trong quá khứ
- 7. Múi giờ và giờ mùa hè
- 8. Định dạng dài và định dạng rộng
- 9. pandas 2 và pandas 3 — những chỗ khác nhau
- 10. polars
- 11. DuckDB
- 12. Bảng đối chiếu nhanh
- 13. Bẫy thường gặp
- Nguồn

1. Môi trường của một buổi

Môi trường là một thư mục chứa đúng các thư viện (đúng phiên bản) mà buổi đó cần. Mỗi buổi có môi trường riêng trong `lab/00-nen/.venv`, dựng bằng lệnh `make up` (xem hướng dẫn cài đặt `MOI-TRUONG.md`). Để chạy một file Python bằng môi trường đó, đứng ở thư mục `lab/` của buổi rồi gõ:

```
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/vi_du.py
```

Đọc lệnh từ trái sang phải:

- `env -u VIRTUAL_ENV`: bỏ qua môi trường Python khác đang bật (nếu có), để khởi chạy nhầm.
- `uv run`: chạy lệnh phía sau bằng Python của một môi trường do công cụ `uv` quản lý.
- `--no-sync`: không cài thêm hay cập nhật gì, dùng đúng những gì đã cài.
- `--project 00-nen`: môi trường nằm ở thư mục `00-nen`.
- `python ../code/vi_du.py`: file cần chạy.

File code dạng percent. Code của khoá là file `.py` bình thường, chia ô bằng chú thích đặc biệt. `# %%` mở một ô code; `# %% [markdown]` mở một ô chữ. Công cụ jupyter đọc các dấu này để mở file như một notebook Jupyter. File vẫn chạy được như script thường, và git so sánh được từng dòng.

```
# %% [markdown]
# # Tiêu đề notebook

# %%
import pandas as pd
```

Gói `tv`. Mỗi môi trường có sẵn gói trợ giúp `tv` của khoá (`import tv`). Ví dụ `tv.THU_MUC_DU_LIEU` là đường dẫn tới thư mục dữ liệu đã kiểm tra, dùng thay cho đường dẫn gõ tay.

2. NumPy tối thiểu

Mảng. NumPy làm việc với **mảng** (array): một dãy số cùng kiểu. Khác list của Python, phép tính trên mảng áp dụng cho **từng phần tử** mà không cần vòng `for`.

```
a = np.array([7, 5, 9, 6, 8])
a * 2          # array([14, 10, 18, 12, 16])  - nhân từng số với 2
a + 1          # array([ 8,  6, 10,  7,  9])
a > 6          # array([ True, False,  True, False,  True])
a[a > 6]       # array([7, 9, 8])              - chỉ giữ các số lớn hơn 6
a.mean()       # 7.0                          - trung bình
```

Dòng `a[a > 6]` gọi là **lọc bằng điều kiện**: `a > 6` cho một mảng True/False, đặt nó trong ngoặc vuông thì chỉ giữ các vị trí True.

Giá trị thiếu. NumPy và pandas ghi giá trị thiếu là `np.nan` (viết tắt của "not a number", không phải số). Hai thư viện xử lý khác nhau. Trong **NumPy**, mọi phép tính có `nan` đều ra `nan`, trừ khi dùng hàm bỏ qua nó. Trong **pandas**, `.sum()` và `.mean()` **mặc định bỏ qua** `nan`, và tổng của một nhóm toàn `nan` ra 0 (mục 4 có cách sửa bằng `min_count`):

```
b = np.array([7.0, np.nan, 9.0])
b.mean()          # nan  - một ô thiếu làm hỏng cả trung bình
np.nanmean(b)     # 8.0  - bỏ ô thiếu: (7 + 9) / 2
pd.Series(b).mean() # 8.0  - pandas tự bỏ ô thiếu
pd.Series(b).mean(skipna=False) # nan  - muốn giống NumPy thì tắt bỏ qua
pd.Series([np.nan, np.nan]).sum() # 0.0  - toàn thiếu mà ra 0: dễ nhầm với "tiêu thụ 0"
```

Số ngẫu nhiên có seed. Seed (hạt giống) là con số khởi động bộ sinh số ngẫu nhiên. Cùng seed thì ra đúng cùng dãy số, nên kết quả chạy lại được. Tạo một bộ sinh riêng bằng `np.random.default_rng(seed)`; đừng dùng `np.random.seed(...)`, vì cách cũ đó đặt seed chung cho cả chương trình và dễ bị đoạn code khác làm lệch.

```
rng = np.random.default_rng(42)
rng.normal(size=3).round(4)      # array([ 0.3047, -1.04 ,  0.7505]) - 3 số từ phân phối chuẩn
rng = np.random.default_rng(42)  # tạo lại với cùng seed...
rng.normal(size=3).round(4)      # array([ 0.3047, -1.04 ,  0.7505]) - ...ra đúng 3 số cũ
```

Quantile. `np.quantile(a, q)` trả về quantile mức `q`, với `q` từ 0 tới 1 (0,25 nghĩa là 25%). Quantile mức `q` là mốc mà khoảng `q` phần số liệu nằm dưới hoặc bằng nó. **Cách đếm tay:** xếp tăng dần, lấy số đứng ở vị trí $q \times n$ làm tròn lên (n là số giá trị). Với `[1, 2, 3, 4]` và `q = 0.25`: $0,25 \times 4 = 1$, lấy số thứ 1, được **1**. Mặc định `np.quantile` không đếm như vậy mà **nội suy** giữa hai số kế nhau, nên có thể ra số không có trong dữ liệu. Có 9 cách tính khác nhau (Hyndman & Fan, 1996), chọn bằng tham số `method`:

```
np.quantile([1, 2, 3, 4], 0.25)                # 1.75   mặc định (method="linear")
np.quantile([1, 2, 3, 4], 0.25, method="median_unbiased") # 1.4167
np.quantile([1, 2, 3, 4], 0.25, method="inverted_cdf")   # 1      khớp cách đếm tay
```

Nội suy là lấy một điểm nằm **giữa** hai số kế nhau, theo tỷ lệ. Cách mặc định tính như sau:

1. Xếp tăng dần và đánh số vị trí **từ 0**: số 1 ở vị trí 0, số 2 ở vị trí 1, số 3 ở vị trí 2, số 4 ở vị trí 3.
2. Vị trí cần lấy = $(n - 1) \times q$, với n là số giá trị. Ở đây $(4 - 1) \times 0,25 = 0,75$.
3. Vị trí 0,75 nằm giữa vị trí 0 (số 1) và vị trí 1 (số 2), cách vị trí 0 đúng 0,75 khoảng.
4. Kết quả = $1 + 0,75 \times (2 - 1) = 1,75$.

Cách `median_unbiased` dùng một công thức vị trí khác, phức tạp hơn (chữ "median" ở đây chỉ tên một tính chất thống kê của cách này, không phải trung vị); không cần tính tay. Cách `inverted_cdf` không nội suy mà lấy đúng một số có trong dữ liệu: định nghĩa và cách đếm tay có ở Phụ lục B mục 4. Khi so kết quả với R (một ngôn ngữ thống kê) hay thư viện khác, kiểm `method` trước khi nghi code sai.

3. pandas tối thiểu và mốc thời gian

Series, DataFrame và chỉ mục

pandas có hai kiểu dữ liệu chính:

- **DataFrame**: một bảng, gồm nhiều cột có tên. Giống một trang tính Excel.
- **Series**: **một cột** của bảng. Lấy một cột ra bằng một cặp ngoặc vuông: `df["kwh"]`.

Mỗi dòng có một **nhãn dòng**, gọi là **chỉ mục** (index), in ở cột ngoài cùng bên trái. Mặc định chỉ mục là 0, 1, 2... Mỗi cột có một **kiểu** (dtype): `int64` là số nguyên, `float64` là số thực (có phần thập phân), `str` là chữ.

```
df = pd.DataFrame({"tinh": ["HN", "HN", "HCM"], "kwh": [10, 12, 20]})
#   tinh kwh
# 0   HN   10
# 1   HN   12
# 2  HCM   20
df.dtypes                # tinh -> str, kwh -> int64
```

```
df["kwh"].tolist()           # [10, 12, 20]    - df["kwh"] là một Series
df.loc[1, "kwh"]             # 12            - .loc[nhãn dòng, tên cột]
df.loc[df["kwh"] > 11, "tinh"].tolist()  # ['HN', 'HCM'] - .loc[điều kiện dòng, tên cột]
```

`.loc[dòng, cột]` chọn theo **nhãn**: phần trước dấu phẩy chọn dòng (một nhãn, hoặc một dãy True/False), phần sau chọn cột. Viết `df["kwh"][df["kwh"] > 11]` cũng chọn được, nhưng đó là **hai lần ngoặc vuông** nối nhau: lần đầu chọn cột, lần sau chọn dòng. Mục 9 cho thấy vì sao không được **gán** giá trị theo kiểu hai lần ngoặc này.

Cột số nguyên có ô thiếu sẽ thành số thực. `nan` là một số thực, nên pandas đổi cả cột sang `float64`:

```
pd.Series([1, 2]).dtype      # int64
pd.Series([1, None]).tolist() # [1.0, nan] - kiểu float64: 1 thành 1.0
```

groupby — gộp theo nhóm. `df.groupby("tinh")["kwh"].sum()` chia bảng thành từng nhóm theo giá trị cột `tinh`, rồi cộng cột `kwh` trong mỗi nhóm:

```
df.groupby("tinh")["kwh"].sum()
# tinh
# HCM    20
# HN     22    ← 10 + 12
```

`resample` ở mục 4 là một kiểu `groupby` đặc biệt: nhóm theo khoảng thời gian thay vì theo giá trị một cột.

Timestamp và `pd.to_datetime`

Một **mốc thời gian** trong pandas là một `Timestamp`. Nó biết mình là ngày nào, giờ nào, thứ mấy:

```
t = pd.Timestamp("2024-03-10 19:30")
t.hour, t.dayofweek, t.day_name()  # (19, 6, 'Sunday') - thứ Hai là 0, Chủ nhật là 6
```

Dữ liệu đọc từ CSV thường là **chuỗi chữ** như `"2024-01-05 08:00"`. `pd.to_datetime` đổi chuỗi thành mốc thời gian. Sau đó dùng `.dt` để lấy từng phần:

```
s = pd.to_datetime(pd.Series(["2024-01-05 08:00", "2024-01-06 09:30"]))
s.dt.hour.tolist()          # [8, 9]
s.dt.dayofweek.tolist()     # [4, 5] - thứ Sáu, thứ Bảy
```

Ba tham số hay cần:

- `format`: nói rõ chuỗi viết theo mẫu nào, ví dụ `format="%Y-%m-%d %H:%M"` (năm-tháng-ngày giờ:phút). Nói rõ thì nhanh hơn và không đoán nhầm.
- `dayfirst=True`: chuỗi viết ngày trước tháng, kiểu Việt Nam. Thiếu nó, `"05/01/2024"` bị hiểu là **1 tháng 5**:

```
pd.to_datetime("05/01/2024")          # 2024-05-01 - hiểu là tháng 5, ngày 1
pd.to_datetime("05/01/2024", dayfirst=True) # 2024-01-05 - ngày 5, tháng 1
```

- `errors="coerce"`: chuỗi nào không đọc được thì thành `NaT` (giá trị thời gian bị thiếu, giống `nan`), thay vì báo lỗi dừng chương trình:

```
pd.to_datetime(pd.Series(["2024-01-05", "không rõ"]))
# ValueError: time data "không rõ" doesn't match format...
pd.to_datetime(pd.Series(["2024-01-05", "không rõ"]), errors="coerce").tolist()
# [Timestamp('2024-01-05 00:00:00'), NaT]
```

Dùng `errors="coerce"` xong thì luôn đếm xem có bao nhiêu `NaT`, để không mất dữ liệu âm thầm: `.isna()` cho True ở ô thiếu, `.sum()` đếm số True, nên `s.isna().sum()` là số ô thiếu.

pd.date_range và bí danh tần suất

`pd.date_range(bat_dau, periods=n, freq=...)` tạo `n` mốc thời gian cách đều nhau. **Tần suất** (khoảng cách giữa hai mốc liền nhau) viết bằng một **bí danh** ngắn:

Bí danh	Nghĩa	Ví dụ 3 mốc đầu từ 1/1/2024
<code>s, min, h</code>	giây, phút, giờ	<code>h</code> : 00:00, 01:00, 02:00
<code>15min</code>	15 phút (số đứng trước là bội số)	00:00, 00:15, 00:30
<code>D</code>	ngày	1/1, 2/1, 3/1
<code>W-MON</code>	tuần, mốc là thứ Hai	1/1, 8/1, 15/1
<code>W</code>	tuần, mốc là Chủ nhật (giống <code>W-SUN</code>)	7/1, 14/1, 21/1
<code>MS</code>	tháng, mốc là ngày đầu tháng	1/1, 1/2, 1/3
<code>ME</code>	tháng, mốc là ngày cuối tháng	31/1, 29/2, 31/3
<code>QE, YE</code>	quý, năm, mốc là ngày cuối kỳ	31/3, 30/6, 30/9 (quý)

```
pd.date_range("2024-01-01", periods=3, freq="MS")
# DatetimeIndex(['2024-01-01', '2024-02-01', '2024-03-01'], dtype='datetime64[us]', freq='MS')
pd.date_range("2024-01-01", periods=3, freq="ME")
# DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31'], dtype='datetime64[us]', freq='ME')
```

`dtype='datetime64[us]'` nghĩa là pandas lưu mỗi mốc bằng số **microsecond** (một phần triệu giây). Mục 9 nói vì sao chi tiết này quan trọng.

Series có chỉ mục thời gian

Một chuỗi thời gian trong pandas thường là một `Series` mà **chỉ mục** (index, cột nhãn bên trái) là các mốc thời gian. Có chỉ mục thời gian thì cắt theo ngày tháng bằng `.loc` rất gọn:

```
y = pd.Series([5, 7, 6, 9, 8, 10], index=pd.date_range("2024-01-01", periods=6, freq="D"))
y.loc["2024-01-03":"2024-01-04"] # 2024-01-03 → 6, 2024-01-04 → 9 (lấy CẢ hai đầu)
y.loc["2024-01"].sum() # 45 – cộng mọi ngày của tháng 1/2024
```

Chú ý: cắt theo ngày tháng bằng `.loc` lấy **cả** mốc cuối. Cắt list Python thì bỏ mốc cuối: với `a = [10, 11, 12, 13, 14]`, `a[2:4]` ra `[12, 13]`, tức lấy chỉ số 2 và 3, bỏ `a[4]`.

4. resample — đổi tần suất

Nó làm gì. `resample` gộp dữ liệu theo từng khoảng thời gian dài hơn, ví dụ từ giờ lên ngày. Sau `resample(...)` phải nói gộp bằng cách nào: `.sum()` (cộng), `.mean()` (trung bình), `.last()` (lấy giá trị cuối)...

Cộng hay trung bình? Tùy loại đại lượng:

- **Lượng tích lũy** (kWh, số chuyến xe, số đơn hàng): **cộng**. 24 giờ, mỗi giờ 2 kWh, thì cả ngày là 48 kWh.
- **Trạng thái tại một lúc** (nhiệt độ, giá, số người đang online): **trung bình** hoặc giá trị cuối. Cộng nhiệt độ 24 giờ không có nghĩa gì.

```
y = pd.Series(range(6), index=pd.date_range("2024-01-01 00:00", periods=6, freq="h")) # 0, 1, ..., 5
lúc 00:00...05:00
y.resample("2h").sum()
# 2024-01-01 00:00:00    1    ← 0 + 1 (giờ 00 và 01)
# 2024-01-01 02:00:00    5    ← 2 + 3
# 2024-01-01 04:00:00    9    ← 4 + 5

nhiet = pd.Series([20.0, 22, 24, 26], index=pd.date_range("2024-01-01", periods=4, freq="h"))
nhiet.resample("2h").mean().tolist() # [21.0, 25.0] — nhiệt độ lấy trung bình
```

Nhãn và biên của khoảng: label và closed. Giá trị gắn nhãn 02:00 là tổng của 00:00–02:00 hay 02:00–04:00? Hai tham số quyết định:

- `closed`: khoảng **gồm** mốc đầu (`"left"` , khoảng [02:00, 04:00), gồm 02:00, bỏ 04:00) hay gồm mốc cuối (`"right"` , khoảng (02:00, 04:00], bỏ 02:00, gồm 04:00).
- `label`: nhãn ghi bằng mốc đầu (`"left"`) hay mốc cuối (`"right"`) của khoảng.

Mặc định cả hai là `"left"` với hầu hết tần suất. **Ngoại lệ:** tần suất gắn với cuối kỳ (`ME` , `QE` , `YE` , `W...`) mặc định `"right"` . Cùng dữ liệu trên, đổi sang `"right"` :

```
y.resample("2h", closed="right", label="right").sum()
# 2024-01-01 00:00:00    0    ← khoảng (22:00 hôm trước, 00:00]: chỉ có giờ 00
# 2024-01-01 02:00:00    3    ← (00:00, 02:00]: giờ 01 + giờ 02 = 1 + 2
# 2024-01-01 04:00:00    7    ← 3 + 4
# 2024-01-01 06:00:00    5    ← chỉ có giờ 05
```

Dữ liệu điện hay ghi "giờ kết thúc" (nhãn 01:00 là điện dùng từ 00:00 tới 01:00). Khi đó `"right"` mới đúng nghĩa. Đọc tài liệu của nguồn dữ liệu trước khi chọn.

ME và MS. Cùng gộp theo tháng, chỉ khác nhãn:

```
d = pd.Series([1.0, 2, 3, 4],
               index=pd.to_datetime(["2024-01-30", "2024-01-31", "2024-02-01", "2024-02-02"]))
d.resample("ME").sum() # 2024-01-31 → 3.0, 2024-02-29 → 7.0
d.resample("MS").sum() # 2024-01-01 → 3.0, 2024-02-01 → 7.0
```

min_count — đừng để khoảng trống thành số 0. Mặc định `.sum()` của khoảng không có số liệu nào ra **0**. Với lượng điện, "0 kWh" và "không đo được" rất khác nhau. `min_count=k` nghĩa là khoảng nào có ít hơn `k` giá trị thật thì để trống (`nan`):

```
z = pd.Series([1.0, np.nan, np.nan, np.nan], index=pd.date_range("2024-01-01", periods=4, freq="h"))
z.resample("2h").sum().tolist()          # [1.0, 0.0] - khoảng thứ hai toàn nan mà ra 0
z.resample("2h").sum(min_count=1).tolist() # [1.0, nan] - đúng: không có số liệu
```

Tự kiểm tra. Chuỗi 1, 2, 3, 4 lúc 00:00, 01:00, 02:00, 03:00. Đoán kết quả của `resample("2h").sum()` và `resample("2h", closed="right", label="right").sum()`.

Đáp án (*tự làm trước rồi mới đọc*)

Mặc định ("`left`"): khoảng [00:00, 02:00) gồm giờ 00 và 01, được $1 + 2 = 3$, nhãn 00:00; khoảng [02:00, 04:00) được $3 + 4 = 7$, nhãn 02:00. Kết quả 3, 7.

Với "`right`": khoảng (22:00, 00:00] chỉ có giờ 00, được 1, nhãn 00:00; (00:00, 02:00] gồm giờ 01 và 02, được $2 + 3 = 5$, nhãn 02:00; (02:00, 04:00] chỉ có giờ 03, được 4, nhãn 04:00. Kết quả 1, 5, 4. Nhầm hay gặp là giữ nguyên số khoảng, chỉ đổi nhãn.

5. Dời và cửa sổ: `shift`, `diff`, `rolling`, `ewm`

Các hàm này tạo **feature** (đặc trưng đầu vào cho mô hình) từ chính chuỗi, như "giá trị giờ trước" hay "trung bình 3 giờ gần nhất". Câu hỏi quan trọng nhất với mỗi hàm: **tại thời điểm t , nó dùng dữ liệu của những thời điểm nào?** Nếu nó dùng dữ liệu **sau** lúc ra dự báo, mô hình được "nhìn trộm" tương lai. Lỗi đó gọi là **rò rỉ tương lai**: điểm đánh giá đẹp giả tạo, rồi hỏng khi dùng thật.

Dùng chung một chuỗi 6 giờ, giá trị 0, 1, 2, 3, 4, 5:

```
x = pd.Series([0.0, 1, 2, 3, 4, 5], index=pd.date_range("2024-01-01", periods=6, freq="h"))
```

`shift` — dời chuỗi

`shift(1)` đẩy mọi giá trị xuống 1 bước: tại giờ t ta thấy giá trị của giờ $t - 1$ (gọi là **trễ 1**). Ô đầu không có giờ trước nên thành `nan`. `shift(-1)` đẩy ngược lên, tức lấy giá trị của **giờ sau**, là tương lai.

```
x.shift(1).tolist()      # [nan, 0.0, 1.0, 2.0, 3.0, 4.0] - trễ 1: an toàn
x.shift(-1).tolist()     # [1.0, 2.0, 3.0, 4.0, 5.0, nan] - giờ sau: KHÔNG được dùng làm feature
```

`diff` — chênh lệch với bước trước

`diff()` tính $y_t - y_{t-1}$: giá trị hiện tại trừ giá trị ngay trước.

```
x.diff().tolist()       # [nan, 1.0, 1.0, 1.0, 1.0, 1.0]
```

Nó dùng y_t , nên chỉ làm feature được nếu lúc ra dự báo đã biết y_t . Nếu y_t chính là thứ cần dự báo, dùng `x.diff().shift(1)`.

`rolling` — cửa sổ trượt

`rolling(3).mean()` tính trung bình của **3 giá trị gần nhất, tính cả giá trị hiện tại**. Cửa sổ trượt dần dọc chuỗi. Hai ô đầu chưa đủ 3 giá trị nên là `nan`.

```
x.rolling(3).mean().tolist() # [nan, nan, 1.0, 2.0, 3.0, 4.0]
```

Tính tay ô thứ 3 (giờ 02:00): $(0 + 1 + 2) / 3 = 1,0$.

- `min_periods=1`: tính luôn khi mới có ít hơn 3 giá trị, thay vì để `nan`.
- `center=True`: đặt cửa sổ **giữa** t , tức dùng y_{t-1} , y_t , y_{t+1} . Có y_{t+1} là tương lai.
- Muốn cửa sổ chỉ gồm quá khứ trước t : thêm `.shift(1)`.

```
x.rolling(3, min_periods=1).mean().tolist() # [0.0, 0.5, 1.0, 2.0, 3.0, 4.0]
x.rolling(3, center=True).mean().tolist()   # [nan, 1.0, 2.0, 3.0, 4.0, nan]
                                             # ô giờ 01 dùng cả giờ 02
x.rolling(3).mean().shift(1).tolist()      # [nan, nan, nan, 1.0, 2.0, 3.0]
                                             # chỉ dùng 3 giờ TRƯỚC t
```

Cửa sổ theo khoảng thời gian. Viết `rolling("2h")` thay vì `rolling(2)` thì cửa sổ là "mọi giá trị trong 2 giờ vừa qua", không phải "2 giá trị gần nhất". Hai cách khác nhau khi dữ liệu có chỗ hỏng:

```
g = pd.Series([1.0, 2, 3],
               index=pd.to_datetime(["2024-01-01 00:00", "2024-01-01 01:00", "2024-01-01 05:00"]))
g.rolling(2).sum().tolist() # [nan, 3.0, 5.0]
                           # ô 05:00 cộng cả giá trị lúc 01:00, cách 4 giờ
g.rolling("2h").sum().tolist() # [1.0, 3.0, 3.0]
                              # ô 05:00 chỉ còn chính nó, vì 01:00 đã quá 2 giờ
```

Vì sao ô đầu của `rolling("2h")` đã có số, còn `rolling(2)` để `nan`? Cửa sổ đếm theo **số giá trị** mặc định đòi đủ số giá trị (`min_periods` bằng độ rộng cửa sổ). Cửa sổ theo **thời gian** mặc định `min_periods=1`: có một giá trị là tính. Muốn đòi ít nhất 2 giá trị thì ghi rõ, ví dụ trên 3 giờ đầu của `x`: `x.iloc[:3].rolling("2h", min_periods=2).sum().tolist()` ra `[nan, 1.0, 3.0]`.

`.iloc` chọn theo **vị trí** (dòng thứ 0, 1, 2...), khác `.loc` chọn theo **nhãn**. `x.iloc[:3]` lấy 3 dòng đầu (vị trí 0, 1, 2) và, giống list Python, **không** lấy vị trí cuối 3. Ngược lại `.loc` với mốc thời gian lấy **cả** mốc cuối (mục 3).

Với cửa sổ thời gian, `closed` quyết định hai đầu cửa sổ có được tính không. Mặc định `"right"`: khoảng $(t - 2h, t]$, bỏ mốc cách đúng 2 giờ, gồm t .

```
x.rolling("2h").sum().tolist() # [0.0, 1.0, 3.0, 5.0, 7.0, 9.0] (t-2h, t]
x.rolling("2h", closed="both").sum().tolist() # [0.0, 1.0, 3.0, 6.0, 9.0, 12.0] [t-2h, t]
x.rolling("2h", closed="left").sum().tolist() # [nan, 0.0, 1.0, 3.0, 5.0, 7.0] [t-2h, t)
                                             # không gồm t
```

Tính tay ô giờ 03:00 với `"both"`: gồm giờ 01, 02, 03, tổng $1 + 2 + 3 = 6$. Với mặc định bỏ giờ 01: $2 + 3 = 5$.

Vì sao ô đầu với `closed="left"` là `nan`, trong khi mục 4 nói tổng của khoảng rỗng ra 0? Ở giờ 00:00, cửa sổ `[22:00, 00:00)` không chứa giá trị nào. `rolling` có `min_periods` (ở đây là 1): chưa đủ 1 giá trị thì trả `nan`. `resample(...).sum()` không có `min_periods`, nên mặc định cộng khoảng rỗng thành 0 (trừ khi thêm `min_count`).

ewm — trung bình trượt hàm mũ

`ewm(alpha=...).mean()` là trung bình mà **giá trị càng mới càng nặng ký**. Dạng đệ quy (`adjust=False`):

$$s_t = \alpha y_t + (1 - \alpha) s_{t-1}$$

- y_t : giá trị mới tại t ; s_t : giá trị đã làm trơn tại t ; s_{t-1} : giá trị đã làm trơn ở bước trước.
- α (alpha): trọng số của giá trị mới, từ 0 tới 1. α lớn thì bám sát giá trị mới; nhỏ thì trơn hơn.

Nói bằng lời. Giá trị trơn mới = α phần giá trị mới + $(1 - \alpha)$ phần giá trị trơn cũ. Với chuỗi 1, 2, 3 và $\alpha = 0,5$: $s_1 = 1$; $s_2 = 0,5 \times 2 + 0,5 \times 1 = 1,5$; $s_3 = 0,5 \times 3 + 0,5 \times 1,5 = 2,25$.

```
pd.Series([1.0, 2, 3]).ewm(alpha=0.5, adjust=False).mean().tolist() # [1.0, 1.5, 2.25]
pd.Series([1.0, 2, 3]).ewm(alpha=0.5).mean().round(3).tolist()      # [1.0, 1.667, 2.429]
```

Mặc định `adjust=True` cho số khác ở các bước đầu. Nó lấy trung bình có trọng số của mọi giá trị đã có, trọng số $1, (1 - \alpha), (1 - \alpha)^2 \dots$ từ mới tới cũ. Tính tay bước 3: trọng số 1; 0,5; 0,25 cho 3, 2, 1, nên $(3 + 1 + 0,25) / (1 + 0,5 + 0,25) = 4,25 / 1,75 \approx 2,429$. Càng về sau, hai cách càng gần nhau.

Bảng tổng hợp: hàm nào nhìn trộm tương lai?

Hàm	Tại thời điểm t dùng	Làm feature được không
<code>shift(1)</code>	y_{t-1}	được
<code>shift(-1)</code>	y_{t+1}	không — tương lai
<code>diff()</code>	y_t và y_{t-1}	chỉ khi y_t đã biết lúc ra dự báo
<code>rolling(3).mean()</code>	y_{t-2}, y_{t-1}, y_t	thêm <code>.shift(1)</code> nếu y_t là thứ cần dự báo
<code>rolling(3, center=True).mean()</code>	y_{t-1}, y_t, y_{t+1}	không — có tương lai
<code>ewm(alpha=...).mean()</code>	mọi giá trị tới t	thêm <code>.shift(1)</code> nếu y_t là thứ cần dự báo

Đọc bảng. Nhìn cột 2: dòng nào có chỉ số $t + 1$ là dùng tương lai, cấm làm feature. Dòng nào có y_t thì phải hỏi "lúc ra dự báo đã biết y_t chưa?". Nếu y_t là thứ đang cần dự báo thì chưa biết, nên phải dời thêm một bước.

Tự kiểm tra. Chuỗi 10, 20, 30, 40 theo giờ liên tiếp. Ở giờ thứ 4, `rolling(2).mean().shift(1)` ra bao nhiêu? Có dùng giá trị 40 không?

Đáp án (tự làm trước rồi mới đọc)

`rolling(2).mean()` ở giờ thứ 3 là $(20 + 30) / 2 = 25$. `shift(1)` dời nó xuống giờ thứ 4, nên ra **25**, không dùng 40. Kết quả cả chuỗi: nan, nan, 15, 25. Nhầm hay gặp là quên `shift(1)`: khi đó giờ thứ 4 ra $(30 + 40) / 2 = 35$, có dùng chính giá trị 40 đang cần dự báo.

6. merge_asof — ghép với giá trị gần nhất trong quá khứ

Nó làm gì. Ghép hai bảng theo thời gian khi mốc thời gian **không trùng nhau**. Mỗi dòng bảng trái lấy dòng bảng phải có thời điểm **gần nhất, trước hoặc đúng bằng** nó. Ví dụ: đơn hàng lúc 10:07 cần giá

sản phẩm. Giá cập nhật lúc 10:05 và 10:10. Đơn hàng phải lấy giá lúc 10:05, vì lúc 10:07 giá 10:10 chưa tồn tại. Lấy giá 10:10 là **rò rỉ tương lai** (mục 5): dùng thông tin chưa có vào lúc đó.

```
don = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:07"])})
gia = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:05", "2024-01-01 10:10"]), "gia": [100, 105]})
pd.merge_asof(don, gia, on="t")          # t = 10:07, gia = 100 - lấy giá lúc 10:05
pd.merge(don, gia, on="t", how="left")   # t = 10:07, gia = NaN
                                         # ghép thường đòi trùng khít từng giây
```

`pd.merge` là phép **ghép thường**: chỉ ghép hai dòng có giá trị `t` **bằng hệt nhau**. `how="left"` nghĩa là giữ mọi dòng của bảng trái (`don`), dòng nào không tìm được cặp thì để trống. Không có `how="left"` thì đơn hàng bị bỏ luôn.

Các tham số:

- `on="t"`: tên cột thời gian dùng để ghép. **Cả hai bảng phải xếp tăng dần** theo cột này, nếu không sẽ báo `ValueError: right keys must be sorted (hoặc left keys...)`.
- `direction="backward"` (mặc định): chỉ nhìn về quá khứ. `"forward"` nhìn về tương lai, dễ gây rò rỉ.
- `allow_exact_matches=False`: không lấy dòng có thời điểm **trùng đúng**, chỉ lấy dòng trước đó. Dùng khi giá trị tại đúng thời điểm `t` chưa kịp công bố lúc ra dự báo.
- `tolerance=pd.Timedelta("1min")`: chỉ ghép nếu dòng quá khứ cách không quá 1 phút; xa hơn thì để trống.

```
trai = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:05"])})
phai = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:03"]), "v": [1, 2]})
pd.merge_asof(trai, phai, on="t")["v"].tolist()          # [1, 2]
pd.merge_asof(trai, phai, on="t", allow_exact_matches=False)["v"].tolist()  # [nan, 2.0]
pd.merge_asof(trai, phai, on="t", tolerance=pd.Timedelta("1min"))["v"].tolist() # [1.0, nan]
```

Lệnh thứ 2: dòng 10:00 bên trái trùng khít với dòng 10:00 bên phải, nhưng bị cấm lấy trùng, và trước đó không còn gì, nên `nan`. Lệnh thứ 3: dòng 10:05 cách 10:03 hai phút, quá giới hạn 1 phút, nên `nan`. Cột `v` có ô `nan` nên thành số thực: 1 thành 1.0 (mục 3).

Tự kiểm tra. Bảng trái có một dòng lúc 10:02. Bảng phải có 10:00 ($v = 1$), 10:02 ($v = 2$), 10:04 ($v = 3$). `merge_asof` mặc định cho `v` bằng mấy? Thêm `allow_exact_matches=False` thì sao?

Đáp án (tự làm trước rồi mới đọc)

Mặc định lấy dòng gần nhất **trước hoặc đúng bằng** 10:02, tức 10:02, nên $v = 2$. Cấm lấy trùng thì lùi về 10:00, $v = 1$. Không bao giờ lấy 3, vì 10:04 nằm sau 10:02 (trừ khi đặt `direction="forward"`, tức nhìn về tương lai).

7. Múi giờ và giờ mùa hè

Quy tắc của khoá. Lưu mọi mốc thời gian theo **UTC** (giờ chuẩn quốc tế, không đổi theo mùa). Chỉ đổi sang giờ địa phương khi hiển thị, hoặc khi tính feature lịch như "giờ trong ngày", "thứ trong tuần".

Độ lệch so với UTC viết sau giờ: `+07:00` nghĩa là giờ địa phương **nhANH hơn** UTC 7 giờ (Việt Nam: 7:00 sáng ở Hà Nội là 0:00 UTC). `-05:00` nghĩa là **chẬM hơn** UTC 5 giờ (New York mùa đông).

Vì sao phải thống nhất múi giờ. Ví dụ bảng chuyển taxi New York ghi giờ địa phương, còn bảng mua ghi giờ UTC, cả hai đều không ghi múi giờ. Mùa hè New York chậm hơn UTC 4 giờ. Mua bắt đầu lúc 21:00 UTC, tức 17:00 ở New York, và chuyển xe tăng vọt đúng lúc 17:00 New York. Ghép thẳng hai bảng theo con số giờ thì cơn mưa hiện ở dòng 21:00, sau đợt tăng chuyển 4 giờ. Kết luận sai: "chuyển xe tăng trước khi mưa". Đổi cả hai về UTC trước khi ghép thì hết.

Naive và aware

- Mốc **naive** không ghi múi giờ: `2024-07-01 12:00`. Không biết đó là 12 giờ ở Hà Nội hay ở London.
- Mốc **aware** có ghi múi giờ: `2024-07-01 12:00+07:00`.
- `tz_localize(múi_giờ)`: **gắn** múi giờ cho mốc naive, **giữ nguyên** con số giờ.
- `tz_convert(múi_giờ)`: **đổi** một mốc aware sang múi giờ khác. Thời điểm thật không đổi, con số giờ đổi.

```
t = pd.Timestamp("2024-07-01 12:00")           # naive
ta = t.tz_localize("Asia/Ho_Chi_Minh")          # 2024-07-01 12:00:00+07:00
                                                # "12 giờ ở Việt Nam"
ta.tz_convert("UTC")                             # 2024-07-01 05:00:00+00:00
                                                # cùng lúc đó, ở UTC là 5 giờ
t.tz_localize("UTC")                             # 2024-07-01 12:00:00+00:00
                                                # "12 giờ UTC", một thời điểm KHÁC
t < ta
# TypeError: Cannot compare tz-naive and tz-aware timestamps
```

Tên múi giờ lấy từ **cơ sở dữ liệu múi giờ IANA**: danh sách chuẩn quốc tế ghi mọi lần một nơi đổi giờ trong lịch sử. IANA là tổ chức quốc tế quản lý tên miền, địa chỉ Internet và danh sách này. Viết theo dạng `Châu_lục/Thành_phố`: `Asia/Ho_Chi_Minh`, `America/New_York`, `Europe/London`.

Feature lịch phải tính theo giờ địa phương. 17:00 UTC ngày 1/1 là 0:00 ngày 2/1 ở Việt Nam:

```
u = pd.Series(pd.to_datetime(["2024-01-01 17:00", "2024-01-01 18:00"])).dt.tz_localize("UTC")
u.dt.hour.tolist()                               # [17, 18] – giờ UTC
u.dt.tz_convert("Asia/Ho_Chi_Minh").dt.hour.tolist() # [0, 1] – giờ Việt Nam, đã sang ngày 2/1
```

Việt Nam không phải lúc nào cũng UTC+7. Hiện nay Việt Nam là UTC+7 và không đổi giờ theo mùa. Nhưng theo cơ sở dữ liệu IANA, điều đó chỉ đúng từ 13/6/1975; trước đó có những giai đoạn lịch khác:

```
from datetime import datetime
import zoneinfo
zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh").utcoffset(datetime(1970, 1, 1)) # 8:00:00
                                                                    # năm 1970 là UTC+8
zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh").utcoffset(datetime(2024, 1, 1)) # 7:00:00
```

Vì vậy đừng tự cộng 7 giờ; hãy dùng `tz_localize/tz_convert` với tên múi giờ.

Giờ mùa hè (DST)

Một số nước (Mỹ, châu Âu) **vận đồng hồ nhanh 1 giờ** vào mùa xuân và vận lại vào mùa thu. Việc này gọi là giờ mùa hè, viết tắt DST (daylight saving time). Hệ quả với dữ liệu theo giờ:

- Ngày vận nhanh có **23 giờ**: một giờ **bị mất**. Ở New York ngày 10/3/2024 không có 02:00–02:59.

- Ngày vắn lại có **25 giờ**: một giờ **bị lặp**. Ngày 3/11/2024 có hai lần 01:00–01:59.

```
pd.date_range("2024-03-10 00:00", periods=4, freq="h", tz="America/New_York")
# 00:00-05:00, 01:00-05:00, 03:00-04:00, 04:00-04:00 ← không có 02:00
pd.date_range("2024-11-03 00:00", periods=4, freq="h", tz="America/New_York")
# 00:00-04:00, 01:00-04:00, 01:00-05:00, 02:00-05:00 ← 01:00 xuất hiện hai lần
```

Phần `-05:00`, `-04:00` là độ lệch so với UTC. Vì sao đổi từ `-05:00` sang `-04:00`? Mùa đông New York chậm hơn UTC 5 giờ. Vắn đồng hồ nhanh thêm 1 giờ thì giờ địa phương chỉ còn chậm hơn UTC 4 giờ. Dữ liệu điện của EIA (Cơ quan Thông tin Năng lượng Mỹ) hay dữ liệu taxi New York đều gặp hai ngày này mỗi năm.

Gắn múi giờ cho giờ bị mất hoặc bị lặp sẽ báo lỗi, trừ khi nói rõ cách xử lý:

```
t = pd.Series(pd.to_datetime(["2024-11-03 01:30"])) # giờ bị lặp: là lần thứ nhất hay thứ hai?
t.dt.tz_localize("America/New_York")
# ValueError: Cannot infer dst time from 2024-11-03 01:30:00, try using the 'ambiguous' argument
t.dt.tz_localize("America/New_York", ambiguous=np.array([True])) # 01:30-04:00
# lần thứ nhất (còn giờ mùa hè)
t.dt.tz_localize("America/New_York", ambiguous=np.array([False])) # 01:30-05:00 – lần thứ hai

m = pd.Series(pd.to_datetime(["2024-03-10 02:30"])) # giờ không tồn tại
m.dt.tz_localize("America/New_York")
# ValueError: 2024-03-10 02:30:00 is a nonexistent time due to daylight savings time.
# Try using the 'nonexistent' argument.
m.dt.tz_localize("America/New_York", nonexistent="shift_forward") # 03:00-04:00
# đẩy tới giờ có thật kế tiếp
```

Các lựa chọn:

- `ambiguous` (giờ bị lặp): `"infer"` tự suy từ thứ tự các dòng; `"NaT"` để trống; `"raise"` báo lỗi (mặc định); hoặc một mảng True/False, True là lần thứ nhất (còn giờ mùa hè). Phải là **mảng**, vì Series có thể nhiều dòng: mỗi dòng một True/False. Ví dụ trên chỉ có một dòng nên mảng có một phần tử.
- `nonexistent` (giờ bị mất): `"shift_forward"` đẩy tới giờ kế tiếp; `"shift_backward"` lùi về giờ trước; `"NaT"` để trống; một khoảng thời gian (ví dụ `pd.Timedelta("1h")`) để cộng thêm; `"raise"` báo lỗi (mặc định).

`"infer"` cần các dòng xếp đúng thứ tự, và giờ lặp phải xuất hiện đủ hai lần:

```
seq = pd.Series(pd.to_datetime(["2024-11-03 00:30", "2024-11-03 01:30",
                                "2024-11-03 01:30", "2024-11-03 02:30"]))
seq.dt.tz_localize("America/New_York", ambiguous="infer").tolist()
# [00:30-04:00, 01:30-04:00, 01:30-05:00, 02:30-05:00] – hai lần 01:30 được tách đúng
```

Trên máy thiếu cơ sở dữ liệu IANA (hay gặp trên Windows), Python cần gói `tzdata`. Môi trường của mọi buổi đã có sẵn.

Tự kiểm tra. Một cảm biến ghi `2024-07-01 20:00` theo UTC. Ở Việt Nam đó là mấy giờ, ngày nào? Nếu tính feature "ngày trong tuần" bằng giờ UTC thì sai ở đâu?

Đáp án (tự làm trước rồi mới đọc)

Cộng 7 giờ: **03:00 ngày 2/7** (`pd.Timestamp("2024-07-01 20:00", tz="UTC").tz_convert("Asia/Ho_Chi_Minh")` ra `2024-07-02 03:00:00+07:00`). Tính bằng giờ UTC thì số đo này bị xếp vào thứ Hai 1/7, giờ 20, trong khi ở Việt Nam là 3 giờ sáng thứ Ba 2/7. Feature lịch lịch cả giờ lẫn thứ trong tuần.

8. Định dạng dài và định dạng rộng

Nhiều chuỗi (ví dụ tải điện của nhiều tỉnh) có thể xếp theo hai cách:

- **Dạng rộng**: mỗi chuỗi một cột, mỗi dòng một thời điểm.
- **Dạng dài**: mỗi dòng là **một** quan sát của **một** chuỗi tại **một** thời điểm. Họ thư viện của công ty Nixtla (`statsforecast`, `mlforecast`, `neuralforecast`) đòi dạng dài với đúng ba cột: `unique_id` (tên chuỗi), `ds` (thời điểm), `y` (giá trị).

`melt` đổi rộng thành dài; `pivot` đổi dài thành rộng:

```
rong = pd.DataFrame({"ds": pd.date_range("2024-01-01", periods=2, freq="D"), "HN": [10, 11], "HCM": [20, 21]})
#           ds  HN  HCM
# 0 2024-01-01   10   20
# 1 2024-01-02   11   21

dai = rong.melt(id_vars="ds", var_name="unique_id", value_name="y")[["unique_id", "ds", "y"]]
#  unique_id      ds  y
# 0         HN 2024-01-01  10
# 1         HN 2024-01-02  11
# 2         HCM 2024-01-01  20
# 3         HCM 2024-01-02  21

dai.pivot(index="ds", columns="unique_id", values="y")    # quay lại dạng rộng (cột xếp theo tên: HCM, HN)
```

Trong `melt`: `id_vars` là cột giữ nguyên; tên các cột còn lại (`HN`, `HCM`) vào cột `var_name`; giá trị vào cột `value_name`.

Dạng dài chứa được các chuỗi dài ngắn khác nhau và dễ thêm cột feature. Dạng rộng tiện để vẽ nhiều chuỗi cạnh nhau và tính tương quan giữa các chuỗi.

9. pandas 2 và pandas 3 — những chỗ khác nhau

Chủ đề	pandas 2.3.3	pandas 3.0.5
Gán qua hai lần ngoặc vuông: <code>df["a"][mask] = v</code>	Có tác dụng , kèm <code>FutureWarning</code>	Không có tác dụng , kèm <code>ChainedAssignmentError</code>
Kiểu của cột chữ	<code>object</code> (kiểu chung "đối tượng Python bất kỳ")	<code>str</code>
<code>pd.to_datetime</code> từ chuỗi	<code>datetime64[ns]</code> (nano giây)	<code>datetime64[us]</code> (micro giây)
Bí danh tần suất cũ <code>H</code> , <code>T</code> , <code>M</code>	chạy, kèm <code>FutureWarning</code>	<code>ValueError: Invalid frequency</code>
<code>fillna(method="ffill")</code>	chạy, kèm <code>FutureWarning</code>	<code>TypeError</code> — dùng <code>.ffill()</code>

Chủ đề	pandas 2.3.3	pandas 3.0.5
Kiểu đối tượng múi giờ	của gói pytz	của thư viện chuẩn zoneinfo (pandas không cài kèm pytz nữa)
groupby trên cột phân loại có nhóm rỗng	vẫn in nhóm rỗng (observed=False)	bỏ nhóm rỗng (observed=True)

Bí danh cũ và bí danh mới (bí danh cũ báo ValueError trên pandas 3, đã chạy thử từng cái):

Cũ	H	T	S	L	U	M	Q	Y, A	BM
Mới	h	min	s	ms	us	ME	QE	YE	BME
Nghĩa	giờ	phút	giây	mili giây	micro giây	cuối tháng	cuối quý	cuối năm	ngày làm việc cuối tháng

Đọc bảng. Bảng trên chỉ để tra. Với bảng so sánh hai bản pandas: cột 2 và cột 3 khác nhau theo hai kiểu. Kiểu nguy hiểm là **cùng code, kết quả khác mà không dùng**: dòng gán hai lần ngoặc, đơn vị thời gian, groupby. Kiểu dễ thấy là code cũ **báo lỗi** trên pandas 3: bí danh cũ, fillna(method=...). FutureWarning là cảnh báo "cách viết này sẽ bị bỏ ở bản sau".

groupby và nhóm rỗng. Cột **phân loại** (kiểu category) là cột chữ mà danh sách giá trị được khai sẵn, kể cả giá trị chưa xuất hiện dòng nào. Khi groupby theo cột này, giá trị chưa xuất hiện tạo ra một **nhóm rỗng**. pandas 2 mặc định vẫn in nhóm rỗng (tổng bằng 0); pandas 3 mặc định bỏ nó:

```
c = pd.DataFrame({"loai": pd.Categorical(["ngay_thuong", "ngay_thuong"],
                                         categories=["ngay_thuong", "ngay_le"]), # khai sẵn 2 giá
                  "kwh": [10, 12]}) # nhưng chỉ có dòng "ngay_thuong"

c.groupby("loai", observed=True)["kwh"].sum() # ngay_thuong → 22 (mặc định pandas 3)
c.groupby("loai", observed=False)["kwh"].sum() # ngay_thuong → 22, ngay_le → 0 (mặc định pandas 2.3.3)
```

Cùng một dòng code không ghi observed, pandas 2 ra 2 dòng, pandas 3 ra 1 dòng. Code phía sau đếm số nhóm hay ghép bảng sẽ lệch âm thầm.

Gán giá trị: luôn dùng .loc. pandas 3 bật cơ chế **Copy-on-Write** (chép khi ghi): mọi phần cắt ra từ một bảng đều xử như một **bản sao** độc lập. Vì vậy df["a"] là một bản sao; gán vào bản sao đó không đổi df.

```
df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df["a"][df["b"] > 4] = 100 # pandas 3: ChainedAssignmentError, df KHÔNG đổi
df["a"].tolist() # [1, 2, 3] (pandas 2.3.3: [1, 100, 100] kèm FutureWarning)
df.loc[df["b"] > 4, "a"] = 100 # cách đúng: một lần .loc với [điều kiện dòng, tên cột]
df["a"].tolist() # [1, 100, 100]
```

Dù tên có chữ "Error", ChainedAssignmentError là một **cảnh báo** (warning), không phải lỗi. Chương trình in cảnh báo rồi **chạy tiếp**, chỉ là df không đổi. Vì không dùng nên rất dễ bỏ sót.

Đơn vị thời gian. Máy lưu mỗi mốc thời gian là một số nguyên: số đơn vị thời gian kể từ 0:00 UTC ngày 1/1/1970. pandas 2 đếm bằng nano giây (một phần tỷ giây), pandas 3 đếm bằng micro giây (một phần triệu giây). Đổi mốc thời gian sang số nguyên thì hai bản ra hai số cách nhau 1.000 lần:

```
ts = pd.to_datetime(pd.Series(["2024-01-01"]))
ts.astype("int64").tolist() # pandas 3: [1704067200000000]
# pandas 2.3.3: [1704067200000000000]
ts.dt.as_unit("ns").astype("int64").tolist() # cả hai bản: [1704067200000000000]
# nói rõ đơn vị thì như nhau
```

Viết code chạy được trên cả hai bản:

- gán bằng `df.loc[dieu_kien, "cot"] = gia_tri`;
- dùng bí danh mới (bảng trên);
- dùng `.ffill()` / `.bfill()` (điền ô trống bằng giá trị trước / sau nó);
- không giả định đơn vị nano giây; cần số nguyên thì `.dt.as_unit("ns")` trước;
- `groupby` trên cột phân loại thì ghi rõ `observed=True` hoặc `observed=False`.

10. polars

polars là thư viện bảng dữ liệu khác pandas, xử lý theo cột và chạy nhiều lõi CPU cùng lúc. Nó có **chế độ lazy** (chạy lười): viết cả chuỗi phép tính trước, polars tối ưu toàn bộ rồi mới chạy khi gọi `.collect()`. Đọc file lớn bằng `pl.scan_parquet(...)` là bắt đầu ở chế độ này, hợp với dữ liệu vài chục triệu dòng.

```
import polars as pl

lz = pl.LazyFrame({"a": [1, 2, 3]}).filter(pl.col("a") > 1) # chưa chạy gì
lz.collect()["a"].to_list() # [2, 3] – giờ mới chạy
```

Bốn từ hay gặp trong code polars:

- `pl.col("v")`: "cột tên v", dùng để viết phép tính trên cột đó.
- `.alias("ten")`: đặt tên cho cột kết quả.
- `df.with_columns(...)`: trả về bảng mới có thêm (hoặc thay) các cột tính trong ngoặc.
- `.agg(...)`: sau khi gộp nhóm, nói mỗi nhóm tính gì (tổng, trung bình...).

Ví dụ `d.with_columns((pl.col("v") * 10).alias("v10"))` thêm cột `v10` bằng cột `v` nhân 10.

Gộp theo khoảng thời gian: `group_by_dynamic`, tương đương `resample`. Mặc định `closed="left"`, `label="left"`, và cột thời gian phải xếp tăng dần:

```
d = pl.DataFrame({"t": pd.date_range("2024-01-01", periods=6, freq="h"), "v": [0, 1, 2, 3, 4, 5]})
d.group_by_dynamic("t", every="2h").agg(pl.col("v").sum())
# t: 00:00, 02:00, 04:00; v: 1, 5, 9 – giống ví dụ resample ở mục 4
```

Trễ và cửa sổ: `pl.col("v").shift(1)` và `pl.col("v").rolling_mean(3)`. Ô thiếu hiện là `None`. Khác pandas, polars cho cột số nguyên chứa ô thiếu, nên `tre_1` vẫn là số nguyên (0, 1, 2...), không thành 0.0, 1.0:

```
d.with_columns(pl.col("v").shift(1).alias("tre_1"))["tre_1"].to_list() # [None, 0, 1, 2, 3, 4]
d.with_columns(pl.col("v").rolling_mean(3).alias("tb3"))["tb3"].to_list() # [None, None, 1.0, 2.0, 3.0, 4.0]
```

Ghép quá khứ gần nhất: `join_asof`, tương đương `merge_asof`, mặc định `strategy="backward"`. Hai bảng phải xếp tăng dần theo cột thời gian:

```
q = pl.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:05"])}))
r = pl.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:03"]), "v": [1, 2]})
q.join_asof(r, on="t")["v"].to_list()      # [1, 2] – cùng dữ liệu ví dụ merge_asof ở mục 6
```

Múi giờ — hai hàm dễ nhầm.

- `dt.replace_time_zone(mui_gio)`: gán (hoặc **thay**) nhãn múi giờ, **giữ nguyên con số giờ**, tức đổi luôn thời điểm thật. Giống `tz_localize` của pandas.
- `dt.convert_time_zone(mui_gio)`: giữ nguyên thời điểm thật, đổi con số giờ. Giống `tz_convert`.

```
d.with_columns(pl.col("t").dt.replace_time_zone("UTC")
               .dt.convert_time_zone("Asia/Ho_Chi_Minh"))["t"][0]  # 2024-01-01
07:00:00+07:00                                                    # đúng

d.with_columns(pl.col("t").dt.replace_time_zone("UTC")
               .dt.replace_time_zone("Asia/Ho_Chi_Minh"))["t"][0]  # 2024-01-01
00:00:00+07:00                                                    # lệch 7 giờ
```

Dòng thứ hai không báo lỗi gì; thời điểm đã lệch 7 giờ âm thầm. Tài liệu polars ghi rõ `replace_time_zone` thay đổi cả mốc thời gian bên dưới.

11. DuckDB

DuckDB là cơ sở dữ liệu chạy ngay trong chương trình Python, không cần cài máy chủ. Ta viết truy vấn bằng **SQL** (ngôn ngữ truy vấn bằng) chạy thẳng trên file Parquet hay CSV. Parquet là định dạng file lưu bảng theo cột, nén tốt, đọc nhanh hơn CSV nhiều.

SQL tối thiểu.

- `SELECT t, y FROM bang_y`: lấy cột `t` và `y` từ bảng `bang_y`. `SELECT *` là lấy mọi cột.
- `WHERE y > 5`: chỉ giữ dòng thỏa điều kiện, giống lọc `df[df["y"] > 5]`.
- `ORDER BY t`: xếp dòng theo cột `t`.
- `-- ...`: chú thích, DuckDB bỏ qua phần sau `--` (giống `#` trong Python).
- `TIMESTAMP '2024-01-01 10:07'`: một mốc thời gian viết thẳng trong câu lệnh; `INTERVAL '15 minutes'`: một khoảng thời gian dài 15 phút.

Ví dụ: `SELECT t, y FROM bang_y WHERE y > 5` trên bảng `bang_y` (dùng ở dưới, `y = 5, 7, 6`) ra hai dòng `01:00 → 7` và `02:00 → 6`.

```
SELECT * FROM 'du-lieu.parquet';                                -- đọc file trực tiếp

-- gộp theo khoảng thời gian, tương đương resample
SELECT time_bucket(INTERVAL '15 minutes', TIMESTAMP '2024-01-01 10:07');  -- 10:00
SELECT date_trunc('hour', TIMESTAMP '2024-01-01 10:07');              -- 10:00 (cắt về đầu
giờ)

-- trễ 1 bước, tương đương shift(1): cột y = 5, 7, 6 cho tre_1 = NULL, 5, 7
SELECT t, y, lag(y) OVER (ORDER BY t) AS tre_1 FROM bang_y;
```

```
-- ghép quá khứ gần nhất, tương đương merge_asof (cùng dữ liệu ví dụ mục 6)
SELECT trai.t, phai.v FROM trai ASOF JOIN phai ON trai.t >= phai.t;      -- (10:00, 1), (10:05, 2)
```

ASOF JOIN ... ON trai.t >= phai.t: với mỗi dòng của `trai`, trong các dòng `phai` thỏa điều kiện (thời điểm không sau nó), chỉ lấy **một dòng gần nhất**. Khác phép ghép thường, vốn ghép với **mọi** dòng thỏa điều kiện. Nhắc lại dữ liệu của mục 6: `trai` có 10:00 và 10:05; `phai` có 10:00 ($v = 1$) và 10:03 ($v = 2$). Dòng 10:05 thỏa với cả 10:00 và 10:03, nhưng chỉ lấy 10:03, nên $v = 2$.

Các bảng `bang_y`, `trai`, `phai` ở trên từ đâu ra? Đó là các DataFrame pandas đã có trong chương trình Python. DuckDB tự tìm biến Python cùng tên và đọc nó như một bảng, không cần chép dữ liệu. `trai`, `phai` là hai bảng ở mục 6; `bang_y` tạo như sau:

```
import duckdb
bang_y = pd.DataFrame({"t": pd.date_range("2024-01-01", periods=3, freq="h"), "y": [5, 7, 6]})
duckdb.sql("SELECT t, y, lag(y) OVER (ORDER BY t) AS tre_1 FROM bang_y ORDER BY t").df()
#           t      y  tre_1
# 0 2024-01-01 00:00:00    5   <NA>
# 1 2024-01-01 01:00:00    7     5
# 2 2024-01-01 02:00:00    6     7
```

Đọc câu SQL:

- `lag(y)`: giá trị `y` của dòng **ngay trước**. `OVER (ORDER BY t)` nói "trước" theo thứ tự nào: xếp theo cột `t`.
- `AS tre_1`: đặt tên cột kết quả, giống `.alias` của polars.
- `.df()`: đổi kết quả về DataFrame pandas. `<NA>`, hay `NULL` trong SQL, là ô không có giá trị, giống `nan`. Chú ý cột `tre_1` vẫn là số nguyên (5, 7), không thành 5.0, 7.0 như quy tắc ở mục 3. Lý do: DuckDB trả về kiểu `Int64` (chữ l hoa), một kiểu số nguyên của pandas **cho phép ô thiếu**, ô thiếu hiện là `<NA>`. Kiểu `int64` thường (chữ i thường) không chứa được ô thiếu nên mới phải đổi sang số thực. Cả hai loại ô thiếu đều nhận ra bằng `.isna()`.
- `::VARCHAR` (dùng ở dưới): đổi giá trị sang chữ, để in ra đúng như DuckDB hiển thị.

Cũng có thể mở kết nối riêng: `con = duckdb.connect(); con.sql("...")`.

TIMESTAMPZ không lưu múi giờ. Kiểu thời gian có múi giờ của DuckDB lưu số micro giây kể từ 0:00 UTC 1/1/1970. Khi **hiển thị** và khi **cắt theo ngày/giờ**, DuckDB dùng cài đặt `TimeZone`, mặc định là múi giờ của máy đang chạy. Cùng một truy vấn, hai máy khác múi giờ cho kết quả khác:

```
SET TimeZone = 'Asia/Ho_Chi_Minh';
SELECT date_trunc('day', TIMESTAMPZ '2024-01-01 20:00:00+00')::VARCHAR;  -- '2024-01-02
00:00:00+07'
SET TimeZone = 'UTC';
SELECT date_trunc('day', TIMESTAMPZ '2024-01-01 20:00:00+00')::VARCHAR;  -- '2024-01-01
00:00:00+00'
```

20:00 UTC ngày 1/1 là 3:00 sáng ngày 2/1 ở Việt Nam, nên "ngày" của nó khác nhau tùy cài đặt. Luôn viết `SET TimeZone = '...'` ở đầu script.

Với pandas 3, lấy cột `TIMESTAMPZ` từ DuckDB về pandas cần gói `pytz` (gói múi giờ cũ của Python, trước khi có `zoneinfo`), vì pandas 3 không cài kèm `pytz` nữa. Thiếu thì báo lỗi "Required module 'pytz' failed to import".

12. Bảng đối chiếu nhanh

Việc	pandas	polars	DuckDB
gộp theo khoảng thời gian	<code>resample("h")</code>	<code>group_by_dynamic("t", every="1h")</code>	<code>time_bucket(INTERVAL '1 hour', t)</code>
ghép quá khứ gần nhất	<code>merge_asof(..., direction="backward")</code>	<code>join_asof(..., strategy="backward")</code>	<code>ASOF JOIN ... ON a.t >= b.t</code>
trễ 1 bước	<code>shift(1)</code>	<code>pl.col("y").shift(1)</code>	<code>lag(y) OVER (ORDER BY t)</code>
trung bình 3 bước gần nhất	<code>rolling(3).mean()</code>	<code>pl.col("y").rolling_mean(3)</code>	<code>avg(y) OVER (ORDER BY t ROWS 2 PRECEDING)</code> (cửa sổ = dòng hiện tại + 2 dòng trước; hai dòng đầu vẫn có số, như <code>min_periods=1</code>)
gắn / đổi múi giờ	<code>tz_localize / tz_convert</code>	<code>dt.replace_time_zone / dt.convert_time_zone</code>	<code>SET TimeZone + TIMESTAMPTZ</code>

13. Bẫy thường gặp

Bẫy	Triệu chứng	Sửa
Gán <code>df["a"][mask] = v</code> trên pandas 3	cột không đổi, chỉ có cảnh báo <code>ChainedAssignmentError</code> , code chạy tiếp	<code>df.loc[mask, "a"] = v</code> (mục 9)
Bí danh cũ <code>H, M, T</code>	<code>ValueError: Invalid frequency</code> trên pandas 3	<code>h, ME, min</code> (mục 3)
Giả định mốc thời gian là nano giây	<code>astype("int64")</code> ra số nhỏ hơn 1.000 lần trên pandas 3	<code>.dt.as_unit("ns")</code> trước (mục 9)
Đọc ngày kiểu Việt Nam không có <code>dayfirst=True</code>	05/01 thành 1 tháng 5	<code>dayfirst=True</code> hoặc <code>format=</code> (mục 3)
<code>resample(...).sum()</code> trên khoảng không có số liệu	ra 0 thay vì trống	<code>sum(min_count=1)</code> (mục 4)
<code>rolling(center=True)</code> hay <code>shift(-1)</code> làm feature	backtest (chấm mô hình trên quá khứ như đang dự báo thật) đẹp giả tạo	cửa sổ lùi + <code>shift(1)</code> (mục 5)
<code>rolling(k)</code> trên dữ liệu có chỗ hỏng	cửa sổ trải dài hơn dự định	<code>rolling("2h")</code> theo thời gian (mục 5)
<code>merge_asof</code> với bảng chưa xếp	<code>ValueError: left keys must be sorted</code>	<code>sort_values</code> theo cột thời gian trước (mục 6)
Dữ liệu naive trộn nhiều múi giờ	"chuyến xe tăng trước khi mưa": bảng giờ địa phương ghép với bảng giờ UTC, lệch 4 giờ	đổi về UTC ngay khi đọc (mục 7)
Tự cộng 7 giờ cho Việt Nam	lệch giờ với dữ liệu trước 1975	<code>zoneinfo / tz_convert</code> (mục 7)
Giờ bị mất / lặp khi gắn múi giờ Mỹ, châu Âu	<code>ValueError</code> ở ngày đổi giờ	<code>ambiguous=, nonexistent=</code> (mục 7)

Bẫy	Triệu chứng	Sửa
<code>replace_time_zone</code> thay cho <code>convert_time_zone</code> (polars)	lịch giờ âm thầm	replace đổi nhãn, convert đổi giờ (mục 10)
DuckDB <code>date_trunc</code> trên <code>TIMESTAMPTZ</code>	kết quả khác nhau giữa các máy	<code>SET TimeZone</code> đầu script (mục 11)

Nguồn

Tra cứu ngày 2026-09-17; code chạy lại ngày 2026-09-18. Nhật ký research của khoá ghi trong thư mục công cụ.

- pandas 3.0.0 — What's new: <https://pandas.pydata.org/docs/whatsnew/v3.0.0.html>
- pandas `to_datetime`: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html
- pandas `DataFrame.resample`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.resample.html>
- pandas `DataFrame.rolling`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>
- pandas `DataFrame.ewm`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.ewm.html>
- pandas `merge_asof`: https://pandas.pydata.org/docs/reference/api/pandas.merge_asof.html
- pandas `Series.tz_localize`: https://pandas.pydata.org/docs/reference/api/pandas.Series.tz_localize.html
- NumPy `quantile`: <https://numpy.org/doc/stable/reference/generated/numpy.quantile.html>
- NumPy random `Generator`: <https://numpy.org/doc/stable/reference/random/generator.html>
- Hyndman, R.J. & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician* 50(4), 361–365.
- Python `zoneinfo`: <https://docs.python.org/3/library/zoneinfo.html>
- IANA tz database, tệp `asia`: <https://data.iana.org/time-zones/tzdb/asia>
- polars `join_asof`: https://docs.pola.rs/api/python/stable/reference/dataframe/api/polars.DataFrame.join_asof.html
- polars `group_by_dynamic`, `dt.replace_time_zone`, `dt.convert_time_zone`: <https://docs.pola.rs/api/python/stable/reference/>
- DuckDB ASOF JOIN: https://duckdb.org/docs/current/guides/sql_features/asof_join.html
- DuckDB hàm thời gian: <https://duckdb.org/docs/current/sql/functions/timestamp.html>
- DuckDB `TIMESTAMP` / `TIMESTAMPTZ`: https://duckdb.org/docs/current/sql/data_types/timestamp.html
- jupyter text định dạng percent: <https://jupyter.org/formats/scripts/>